HTTP · RFC 9110 + RFC 9457

API REST bien hecha

Elige el verbo correcto, responde con el código que toca y devuelve errores que el front entiende sin adivinar.



Chamba Lab · 2026



El búho: el cliente que consume tu API

Una API REST es un contrato sobre HTTP

Tú dices qué quieres; el servidor dice cómo terminó.

- Recursos con nombre: `/pedidos/1042`
- Verbos que dicen la intención: `GET`, `POST`, `PATCH`...
- Un código de estado que cualquier programa entiende
- El ejemplo de todo el manual: la API de pedidos de una bodega

Anatomía: una petición y su respuesta



Método

Qué quieres hacer. La intención la pone el cliente.

Recurso

Sobre qué: un sustantivo en plural y su id. Nada de `/verPedido`.

Código

Cómo terminó, en tres dígitos. Lo leen las máquinas.

Cuerpo

El detalle: el pedido en JSON o la explicación del error.

Los verbos: cada uno promete algo distinto

GET

Leer: no cambia nada en el servidor.

`/pedidos`

`/pedidos/1042`

SEGURO · IDEMPOTENTE

POST

Crear: agrega un pedido nuevo; el servidor elige el id.

`/pedidos`

NI SEGURO NI IDEMPOTENTE

PUT · PATCH

Cambiar: PUT reemplaza el pedido entero; PATCH cambia solo una parte.

`/pedidos/1042`

PUT IDEMPOTENTE · PATCH NO

DELETE

Borrar: elimina el recurso.

`/pedidos/1042`

IDEMPOTENTE

Idempotente: repetirlo no cambia el efecto



Como el ascensor: pulsar cinco veces trae uno solo

```
# se cortó la red: el cliente reintenta
DELETE /pedidos/1042   # 204
DELETE /pedidos/1042   # 404
# sigue borrado: mismo efecto

POST /pedidos          # 201, id 1042
POST /pedidos          # 201, id 1043
# dos pedidos: el reintento duplicó
```

POST no es idempotente: reintentar a ciegas crea otro pedido (y otro cobro).

Códigos de estado: el primer dígito ya dice quién falló

1xx

Informativo: la respuesta aún no termina.

`100` `103`

RAROS EN UNA API

2xx

Éxito: salió bien.

`200` `201` `204`

A DIARIO

3xx

Redirección: está en otro lado o no cambió.

`301` `304`

MÁS EN WEBS

4xx

Error del cliente: corrige la petición; repetirla igual no sirve.

`400` `401` `404` `409`

A DIARIO

5xx

Error del servidor: reintentar más tarde puede funcionar.

`500` `503`

QUE SEAN POCOS

Quién lee el código: no solo tu front

El front con fetch
`res.ok = 200 a 299`

Caché y CDN
`200 y 404 se reutilizan`

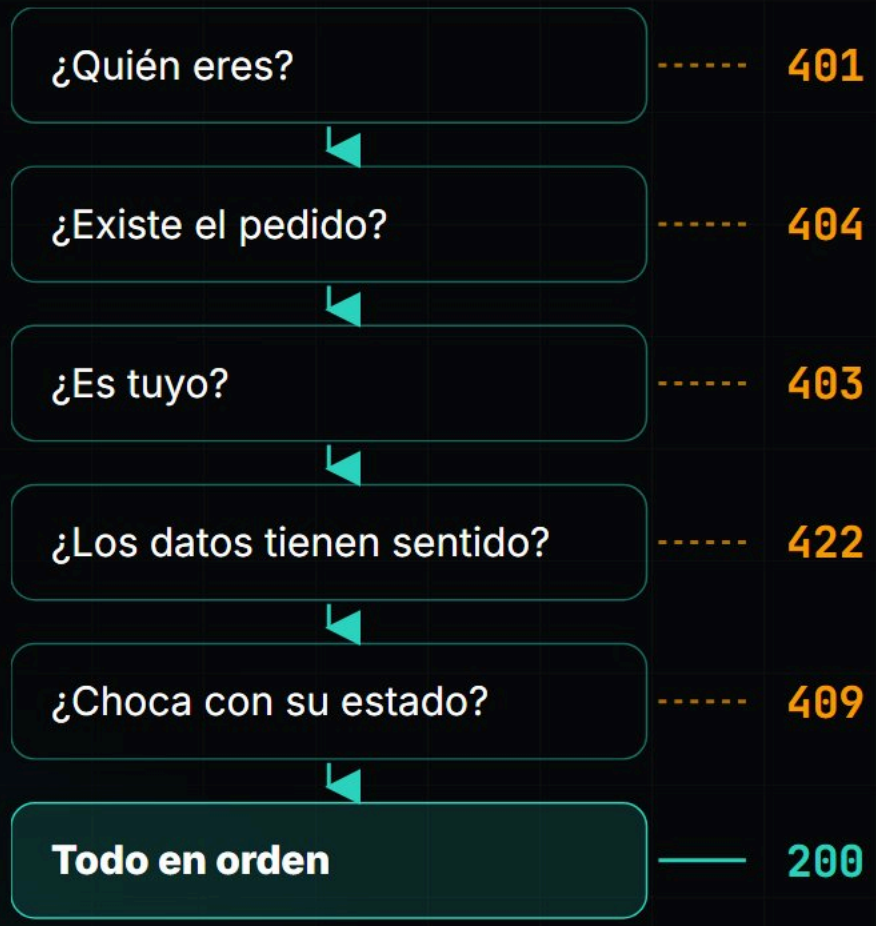
Código de estado

Monitoreo y alertas
`cuentan los 5xx`

Reintentos
`503 + Retry-After`

Ninguno abre tu JSON para saber si salió bien: todos miran los tres dígitos.

Cómo decide el servidor: filtros en orden




```
// el 401 ya lo respondió el middleware
app.patch('/pedidos/:id', (req, res) ⇒ {
  const pedido = buscar(req.params.id);
  if (!pedido) return res.sendStatus(404);
  if (pedido.clienteId ≠= req.usuario.id)
    return res.sendStatus(403);
  if (!ESTADOS.includes(req.body.estado))
    return res.sendStatus(422);
  if (pedido.estado ≡ 'entregado')
    return res.sendStatus(409);
  pedido.estado = req.body.estado;
  res.json(pedido); // 200 por defecto
});
```

Se confunden: mismo síntoma, distinto caso

401 **No sé quién eres:** falta el token o venció. Va con `WWW-Authenticate`.

403 **Sé quién eres,** pero no tienes permiso.

400 **No se entiende:** el JSON llegó mal formado.

422 **Se entiende, pero no vale:** `"estado": "volando"`.

404 **No existe:** no hay pedido 9999.

405 **La ruta existe, ese verbo no.** Va con `Allow`.

500 **Se rompió algo** que el servidor no esperaba.

503 **Saturado o en mantenimiento:** vuelve luego.

¿Pedido ajeno? Puedes responder `404` en vez de `403` para no revelar que existe (RFC 9110 §15.5.4).

Éxito con matices: 200, 201 y 204

pedidos.js · Express 5

```
// crear: 201 y dónde quedó
app.post('/pedidos', (req, res) => {
  const pedido = crear(req.body);
  res.status(201)
    .location(`/pedidos/${pedido.id}`)
    .json(pedido);
});

// borrar: 204, sin cuerpo
app.delete('/pedidos/:id', (req, res) => {
  borrar(req.params.id);
  res.status(204).end();
});
```

`201 Created`: se creó algo nuevo y `Location` dice su URL.

`204 No Content`: hecho, y no hay nada que devolver.

`200 OK`: leer o actualizar, con el recurso en el cuerpo.

PUT o PATCH: dos formas de cancelar un pedido

PUT /pedidos/1042

```
{
  "clienteId": 7,
  "producto": "arroz 5 kg",
  "cantidad": 2,
  "estado": "cancelado"
}
```

Mandas el pedido completo: lo que omites se pierde. Idempotente.

PATCH /pedidos/1042

```
{
  "estado": "cancelado"
}
```

Mandas solo lo que cambia. Formato típico: JSON Merge Patch (RFC 7396).

Errores que se entienden: Problem Details

409 · application/problem+json

```
{
  "type": "https://example.com/errores/ya-entregado",
  "title": "El pedido ya fue entregado",
  "status": 409,
  "detail": "El pedido 1042 se entregó a las 13:05.",
  "instance": "/pedidos/1042"
}
```

`type`: qué error es. El front decide con él; si falta, vale `about:blank`.

`title` es fijo por tipo; `detail` cuenta lo que pasó esta vez.

`status` repite el código, `instance` señala el caso. Puedes sumar campos propios.

En Express: un 422 que dice qué corregir



Un error útil señala la casilla exacta

- `.type()` va antes: `.json()` respeta el tipo ya puesto
- `errors` es una extensión, como en el ejemplo de la RFC

```
app.post('/pedidos', (req, res) ⇒ {
  if (!(req.body.cantidad > 0)) {
    return res.status(422)
      .type('application/problem+json')
      .json({
        type: 'https://example.com/errores/datos',
        title: 'Datos del pedido no válidos',
        status: 422,
        errors: [{ detail: 'debe ser mayor que 0',
                   pointer: '#/cantidad' }],
      });
  }
  // ... crear y responder 201
});
```



La hámster de QA encuentra todos los bugs

¿Y si respondo siempre 200 y pongo el error en el JSON?

Funciona en tu máquina... hasta que QA lo prueba.

El 200 mentiroso: el error viaja como éxito

`fetch` solo falla si se cae la red. Con un 200, `res.ok` es `true` y el error entra como si fuera un pedido.

- La caché puede guardarlo como respuesta buena
- El monitoreo marca 100 % de éxito
- El búho ve una pantalla vacía y sin mensaje

```
// servidor
app.get('/pedidos/:id', (req, res) => {
  const pedido = buscar(req.params.id);
  if (!pedido)
    return res.json({ error: 'No existe' }); // 200
  res.json(pedido);
});

// cliente
const res = await fetch('/pedidos/9999');
if (res.ok) {
  const pedido = await res.json();
  pintar(pedido.estado); // undefined
}
```

Matriz de decisión: cómo responder un error

	200 + error en JSON	Código + JSON propio	Código + Problem Details
fetch, caché y monitoreo lo entienden	No: todo parece éxito	Sí	Sí
El front sabe qué corregir	Solo leyendo el texto	Sí, con un formato por API	Sí: `type` y errores por campo
Formato común entre equipos	No	No: cada API inventa el suyo	Sí: estándar RFC 9457

Código correcto siempre y Problem Details para el cuerpo del error. Si ya tienes tu propio JSON de error, corrige primero los códigos y migra el formato después.

Lo que te llevas

- El verbo dice la intención; el código, cómo terminó
- 4xx: corrige el cliente · 5xx: falló el servidor
- Errores en `application/problem+json`, con `type` y detalle

> rfc: rfc-editor.org/rfc/rfc9110 · HTTP Semantics

> rfc: rfc-editor.org/rfc/rfc9457 · Problem Details

> mdn: developer.mozilla.org · HTTP status codes

> repo: github.com/Chamba-Lab